

HTTP Protocol

Message Types, Syntax, Semantics, and Rules

1 Types of Messages Exchanged

HTTP operates on a simple client-server request/response model. The two primary message types are:

- **Request (client → server):**
 - The client initiates communication by sending an HTTP request to the server.
 - The request line specifies the method (e.g., `GET`, `POST`, `HEAD`), the target resource (URI), and the HTTP version.
 - Following the request line are zero or more header fields, each in the form `Header-Name: value`, each terminated by CRLF.
 - A blank line (CRLF) indicates the end of the header section.
 - Optionally, a message body follows (for methods like `POST` or `PUT`).
- **Response (server → client):**
 - The server replies with an HTTP response.
 - The status line indicates the HTTP version, a status code (e.g., 200, 404, 500), and a reason phrase (e.g., `OK`, `Not Found`, `Internal Server Error`).
 - Following the status line are zero or more header fields (e.g., `Content-Type`, `Content-Length`, `Date`), each terminated by CRLF.
 - A blank line (CRLF) marks the end of the header section.
 - Optionally, the response contains a message body (for 2xx or 4xx/5xx responses with a payload).

2 Message Syntax

Below are the syntactic formats (using CRLF to denote `\r\n`).

2.1 Request Message Format

```
<Method> <Request-URI> HTTP/<Major>.<Minor><CRLF>
<Header-Name-1>: <value-1><CRLF>
<Header-Name-2>: <value-2><CRLF>
    \vdots
<Header-Name-N>: <value-N><CRLF>
<CRLF>
<Optional Request Body>           (length determined by Content-Length header)
```

Example (simple GET request):

```
GET /index.html HTTP/1.1\r\n
Host: www.example.com\r\n
User-Agent: MyBrowser/1.0\r\n
Accept: text/html,application/xhtml+xml\r\n
Connection: keep-alive\r\n
\r\n
```

2.2 Response Message Format

```
HTTP/<Major>.<Minor> <Status-Code> <Reason-Phrase><CRLF>
<Header-Name-1>: <value-1><CRLF>
<Header-Name-2>: <value-2><CRLF>
    \vdots
<Header-Name-N>: <value-N><CRLF>
<CRLF>
<Optional Response Body>          (length determined by Content-Length or Transfer-
    Encoding)
```

Example (200 OK response):

```
HTTP/1.1 200 OK\r\n
Date: Tue, 20 May 2025 12:34:56 GMT\r\n
Server: Apache/2.4.41 (Ubuntu)\r\n
Content-Type: text/html; charset=UTF-8\r\n
Content-Length: 1256\r\n
Connection: keep-alive\r\n
\r\n
<html>
  <head><title>Example</title></head>
  <body>
    <h1>Welcome to Example.com</h1>
    ...
  </body>
</html>
```

3 Message Semantics

Each line and header field carries specific semantic meaning:

3.1 Request Semantics

- **<Method>**
 - GET: Retrieve the resource identified by **<Request-URI>**.
 - POST: Submit data to the server (e.g., form data).
 - HEAD: Same as GET but only fetch headers (no body).
 - Others: PUT, DELETE, OPTIONS, etc.
- **<Request-URI>**
 - The path (and optional query string) identifying the target resource on the origin server.
 - Example: `/path/to/resource.html?user=alice&id=42`
- **HTTP/<Major>.<Minor>**
 - Version of HTTP used (e.g., HTTP/1.0, HTTP/1.1).
 - Indicates feature set (persistent connections, chunked encoding, etc.).
- **Host: <hostname>**
 - Required in HTTP/1.1. Specifies the Internet host and (optionally) port number of the resource being requested.
 - Enables virtual hosting (multiple domains on one IP).
- **User-Agent: <user-agent-string>**
 - Identifies the client software, version, and sometimes platform.
 - Servers can use this for logging, content negotiation, or feature-detection.
- **Accept: <media-ranges>**

- Indicates which media types (MIME types) the client is willing to accept (e.g., `text/html`, `application/json`).
- Servers perform content negotiation based on this.
- **Connection:** `<token-list>`
 - Controls whether the network connection stays open after the current transaction (`keep-alive`) or is closed (`close`).
 - In HTTP/1.1, connections are persistent by default unless “**Connection:** `close`” is specified.
- **Content-Length:** `<length>` (for requests with body)
 - Number of bytes in the request body (e.g., for POST or PUT).
 - Client must send exactly this many bytes following the blank line.

3.2 Response Semantics

- **HTTP/**`<Major>`**.**`<Minor>`
 - Matches the version the server supports. HTTP/1.1 may send chunked encoding.
- **<Status-Code>** (3-digit integer)
 - **1xx** (Informational): e.g., 100 `Continue`.
 - **2xx** (Success): 200 `OK`, 201 `Created`, etc.
 - **3xx** (Redirection): 301 `Moved Permanently`, 302 `Found`, etc.
 - **4xx** (Client Error): 400 `Bad Request`, 401 `Unauthorized`, 404 `Not Found`, etc.
 - **5xx** (Server Error): 500 `Internal Server Error`, 503 `Service Unavailable`, etc.
- **<Reason-Phrase>**
 - Short textual description of the status code.
 - Optional for machine parsing, but improves human readability.
- **Date:** `<RFC1123-date>`
 - The date and time at which the response was originated.
 - Example format: `Tue, 20 May 2025 12:34:56 GMT`.
- **Server:** `<server-software/version>`
 - Identifies the server implementation.
 - Useful for debugging or analytics.
- **Content-Type:** `<media-type>; charset=<encoding>`
 - The MIME type of the response body (e.g., `text/html; charset=UTF-8`, `image/png`).
 - Informs the client how to interpret the bytes in the body.
- **Content-Length:** `<length>`
 - Number of bytes in the response body (unless chunked encoding is used).
 - Client reads exactly this many bytes after the blank line.
- **Transfer-Encoding:** `chunked`
 - In HTTP/1.1, indicates the response is sent in chunks.
 - Each chunk begins with its length in hex followed by CRLF, then the chunk data, then another CRLF.
 - A final zero-length chunk (`0*`) signals end of the body.
- **Connection:** `<token-list>`
 - If **Connection:** `close`, the server will close the TCP connection after this response.
 - If omitted (in HTTP/1.1), the connection is persistent by default.

4 Rules for When and How Processes Send & Respond to Messages

4.1 Client Behavior

1. Establish TCP Connection

- Client opens a TCP socket to the server's IP on port 80 (HTTP) or 443 (HTTPS/TLS).
- If using HTTPS, a TLS handshake occurs first.

2. Send Request

- Client composes a request line:
`<Method> <Request-URI> HTTP/1.1`
- Add mandatory headers:
 - `Host: <hostname>` (required in 1.1)
 - `User-Agent: <client-software/version>`
 - `Accept: <media-ranges>` (optional but common)
 - `Connection: keep-alive` (default in HTTP/1.1) or `Connection: close`
- Optionally add `Content-Length` and `Content-Type` if a body is present.
- Terminate headers with a blank line (CRLF).
- Send the request body (if any) immediately after the blank line.

3. Wait for Server Response

- Client reads the response's status line.
- Client reads response headers until encountering a blank line.
- Client determines body length:
 - If `Transfer-Encoding: chunked`, read chunked data until zero-length chunk.
 - Else read exactly `Content-Length` bytes.
- Client processes the response body (e.g., render HTML, store image).

4. Connection Management

- If `Connection: close` was sent or received, client closes TCP socket.
- If persistent (keep-alive), client may send further requests on the same connection (pipelining or sequential).

4.2 Server Behavior

1. Accept TCP Connection

- Server listens on port 80 (or 443 for HTTPS).
- On incoming TCP connection, optionally perform TLS handshake.

2. Read Client Request

- Parse the request line:
 - Extract `<Method>`, `<Request-URI>`, and HTTP/1.x version.
 - If request line malformed, respond `400 Bad Request` and close connection.
- Read request headers until blank line.
- Validate `Host:` header (must exist in HTTP/1.1).
- If `Content-Length` is present, read that many bytes as request body.

3. Process Request

- Look up the resource at `<Request-URI>`.
- If resource exists and is readable:
 - Generate status line: `HTTP/1.1 200 OK`.
 - Determine `Content-Type` based on file extension or server configuration.

- Determine payload length (or send with chunked encoding).
- Set **Date**, **Server**, and any other relevant headers (**Last-Modified**, **ETag**, etc.).
- If resource not found:
 - Respond with **HTTP/1.1 404 Not Found**, include a short HTML body.
- If the server cannot read or process resource (e.g., file permission, internal error):
 - Respond with **HTTP/1.1 500 Internal Server Error**.

4. Send Response

- Write status line: **HTTP/1.1 <Status-Code> <Reason-Phrase>**.
- Write required headers:
 - **Date:** <current RFC1123-date>
 - **Server:** <server-software/version>
 - **Content-Type:** <mime-type>
 - **Content-Length:** <payload-length>
 - **Connection:** keep-alive (default in HTTP/1.1) or **Connection:** close
- Terminate headers with blank line (CRLF).
- Write response body (if any).

5. Connection Management

- If **Connection:** close is specified, close the TCP connection after sending the response.
- Otherwise (persistent), leave socket open to handle pipelined requests or future requests.

4.3 Error Handling and Redirection

- **400 Bad Request:** Malformed syntax in request line or headers.
- **401 Unauthorized:** Authentication is required (if server enforces).
- **403 Forbidden:** Server understood request but refuses to authorize.
- **404 Not Found:** Requested resource does not exist.
- **500 Internal Server Error:** Generic server-side failure.
- **301 Moved Permanently / 302 Found / 307 Temporary Redirect:**
 - Include **Location:** <new-URL> header to tell the client where to request next.
 - Client should issue a new request to the URL given in **Location**.

5 Summary of Key Points

- HTTP exchanges **request** and **response** messages over TCP (or TLS).
- Each message consists of a start/status line, header fields (terminated by CRLF), a blank line, and an optional body.
- Header fields are always of the form **Name:** value and delimited by CRLF.
- The semantics of headers determine content negotiation (**Accept**), host targeting (**Host**), caching (**If-Modified-Since**, **ETag**), and connection persistence (**Connection**).
- Clients open connections, send requests, parse responses, verify status codes, and optionally reuse persistent connections.
- Servers accept connections, parse requests, validate headers, locate or generate resources, send responses, and close or keep connections alive.
- Error codes inform clients when requests fail; redirection codes guide clients to alternate URLs.